Understanding and Evaluating CPU Scheduling Algorithms in Modern Operating Systems

Introduction

CPU scheduling plays a vital role in managing the execution of processes in an operating system (OS). It ensures that the CPU is utilized effectively by allocating CPU time to processes in the ready queue. The goal is to optimize system performance, minimize delays, and enhance the user experience. Key concepts like CPU burst, turnaround time, waiting time, and response time are critical metrics used to evaluate the efficiency of scheduling algorithms.

- **CPU Burst**: The time a process requires for execution on the CPU.
- **Turnaround Time**: The total time taken for a process to complete, from submission to completion.
- **Waiting Time**: The amount of time a process spends in the ready queue.
- **Response Time**: The time from when a request is made to the first response.

Preemptive vs Non-preemptive Scheduling

- Preemptive Scheduling allows a process to be interrupted and resumed later if a higher-priority process needs the CPU.
- Non-preemptive Scheduling means a process cannot be interrupted once it begins its execution; it runs to completion.

1. Understanding CPU Scheduling Algorithms

First Come First Serve (FCFS)

- **Concept**: This is the simplest scheduling algorithm where the first process to arrive is executed first.

- **Advantages**: Simple to implement, and no starvation.

- **Disadvantages**: The major drawback is the convoy effect, where a long CPU-bound process causes shorter processes to wait unnecessarily, increasing the overall waiting time.

Shortest Job First (SJF)

- **Concept**: This algorithm selects the process with the shortest CPU burst time for execution next.

- **Advantages**: Provides the optimal solution by minimizing average waiting time.

- **Disadvantages**: It is difficult to predict the exact burst time of a process, and it suffers from starvation where longer processes may never get executed.

Priority Scheduling

- **Concept**: Each process is assigned a priority, and the CPU is allocated to the process with the highest priority.

- **Advantages**: Can be used in systems where certain tasks need to be prioritized over others.

- **Disadvantages**: Low-priority processes may starve unless aging is used to increment their priority over time.

Round Robin (RR)

- **Concept**: The CPU is assigned to each process for a fixed time quantum. Once the quantum expires, the process is preempted and the next process in the queue is allocated the CPU.

- **Advantages**: Simple and ensures fair allocation.

- **Disadvantages**: Performance is highly dependent on the size of the time quantum, and if it's too small, it may lead to too many context switches.

Multilevel Queue

- **Concept**: This scheduling algorithm partitions processes into separate queues based on their priority or type (e.g., interactive, batch), and each queue may have its own scheduling algorithm.

- **Advantages**: Helps manage different types of processes efficiently.

- **Disadvantages**: Once a process is assigned to a queue, it generally stays there for the duration of its execution, limiting flexibility.

Multilevel Feedback Queue

- **Concept**: Similar to multilevel queue scheduling, but processes can migrate between queues based on their behavior (e.g., using more CPU time moves them to a lower-priority queue).

- **Advantages**: More flexible than multilevel queues.

- **Disadvantages**: More complex to implement and manage.

2. Algorithm Performance Evaluation

We will compare three algorithms: FCFS, SJF, and RR, using the following metrics: average waiting time, turnaround time, CPU utilization, and fairness.

Hypothetical Process Data:

Process	Arrival Time	Burst Time
P1	0	8
P2	1	4
P3	2	9
P4	3	5

Comparison Table:

Metric	FCFS	SJF	RR (Quantum = 4)
Average Waiting Time	17 ms	6.5 ms	5.66 ms
Average Turnaround Time	28 ms	18 ms	23.33 ms

CPU Utilization	100 %	100 %	100%
Fairness	Low	High	Medium

Explanation:

- FCFS results in high waiting times because long jobs delay the short ones.
- SJF minimizes waiting time but suffers from prediction issues and starvation.
- RR offers a fairer approach, but the average waiting time can be higher depending on the quantum size.

3. Analytical Application

Preemption and Fairness:

Preemption improves system responsiveness by ensuring that no process monopolizes the CPU for too long. It enhances fairness by giving equal time slices to processes, especially in Round Robin and Priority Scheduling.

Starvation:

Starvation occurs when low-priority processes are never allocated to the CPU due to the continuous arrival of higher-priority processes. Priority Scheduling and SJF are especially prone to this issue, but techniques like aging (gradually increasing the priority of waiting processes) can help mitigate it.

4. Real-World OS Case Study Comparison

Linux vs. Windows Scheduling:

- **Linux** uses the **Completely Fair Scheduler (CFS)** which aims to allocate CPU time in proportion to the weight of each process. The system assigns each process a `vruntime` to track how much CPU time it has consumed.

- **Windows** uses a **priority-based preemptive scheduling** algorithm with 32 levels of priority. The dispatcher ensures that the highest-priority thread is always given the CPU.

Differences: While Linux uses a more balanced and fair approach with its CFS, Windows prioritizes processes based on fixed priority levels, which can lead to higher responsiveness for real-time tasks.

5. Critical Thinking Scenario

For a real-time embedded system, Rate-Monotonic Scheduling (RMS) would be most appropriate. It ensures that tasks with the shortest periods (requiring the CPU more frequently) are prioritized. This approach is ideal for systems like a pacemaker where tasks must be completed within strict deadlines to ensure safety and reliability.

Conclusion

CPU scheduling plays a critical role in optimizing system performance, minimizing waiting times, and ensuring fairness. Different algorithms excel in different environments, and it is essential to choose the right one based on the system's specific requirements. Understanding the trade-offs between fairness, efficiency, and responsiveness is key to designing robust systems.